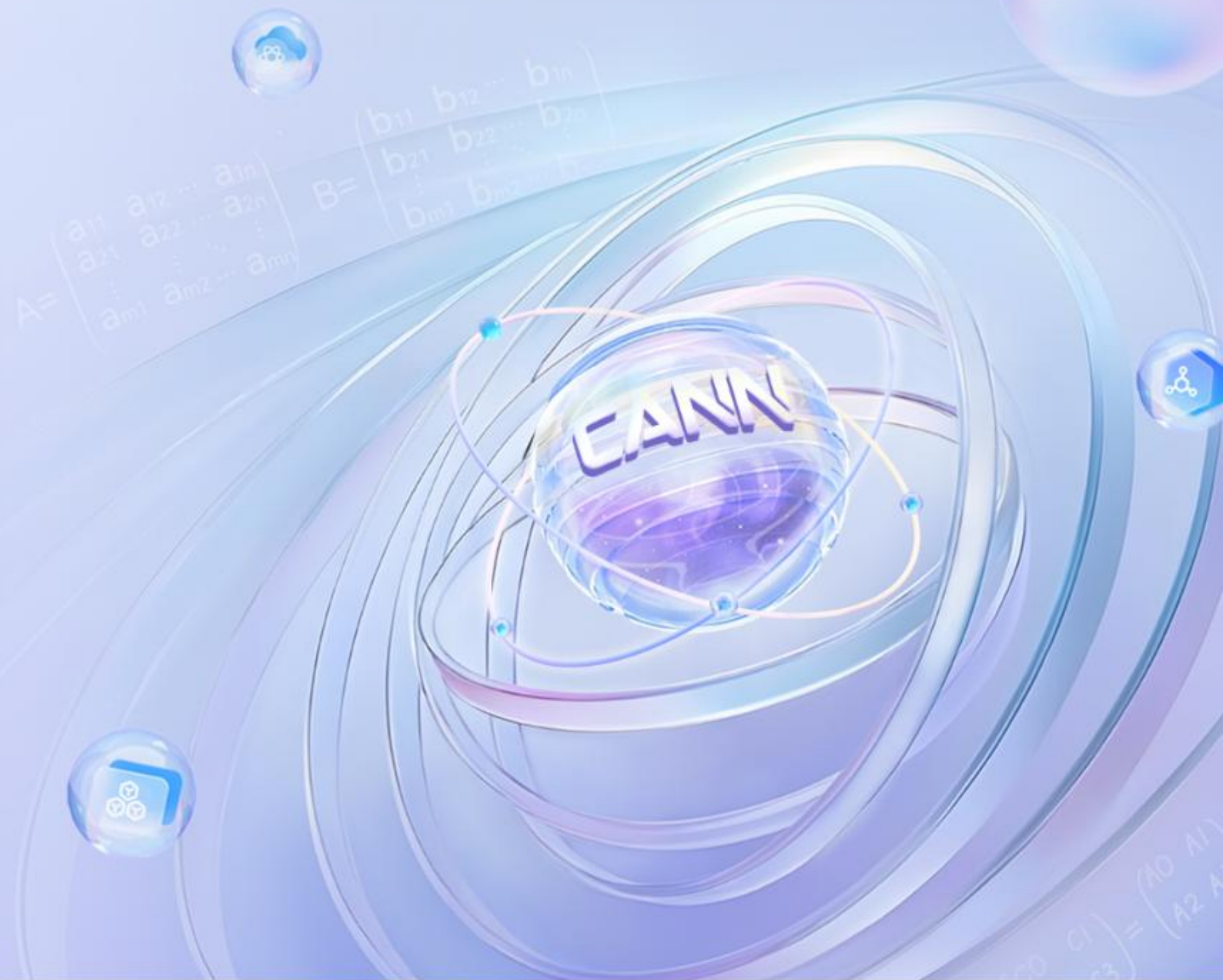


大模型量化基础

昇腾 CANN 启航营

从原理到实践的量化入门



目录

- 01 为什么需要量化**
从推理成本压力看量化的必要性
- 03 W8A8 量化详解**
当前工业界最成熟的量化方案
- 05 CANN 仓库实践**
在 cann-recipes-infer 中落地量化

- 02 量化基础概念**
数值格式、量化位置、量化粒度与量化映射
- 04 主流量化路线对比**
GPTQ、AWQ、FP8 与更多方案

01 为什么需要量化

从推理成本压力看量化的必要性

大模型推理的瓶颈压力

大模型参数量巨大，推理过程中涉及海量矩阵计算，带来显著的显存、带宽和计算压力。

**显存占用**

Memory

Qwen3-235BFP16 权重约 470GB
远超单卡显存容量
长上下文 KVcache也会成为瓶颈

**带宽瓶颈**

Memory BW

推理阶段激活、权重和中间结果
会频繁读写
带宽速度往往跟不上计算速度

**算力不足**

Compute Power

低精度计算能力
是高精度计算能力的数倍
新硬件支持低精度计算

**时延与吞吐**

Latency/Throughput

高吞吐和低时延
是客户最关心的性能指标
“一份性能对应一份价格”



量化——将高精度数值（FP16/FP32）映射到低精度数值（INT8/FP4）
进而通过降低数值比特实现**显存占用减小**、**降低带宽压力**、**算力翻倍**、**高吞吐与低时延**

面对日益增长的模型规模和部署需求，降低推理成本已成为大模型落地的核心课题。量化技术正是解决上述成本压力的核心手段。

02

量化基础概念

数值格式、量化位置、量化粒度与量化映射

数值格式概览

不同数值格式在精度与效率之间做出不同权衡

格式	位数	典型用途	特点
FP32 E8M23	32 bit	训练主权重	精度高，成本高
FP16 E5M10	16 bit	训练/推理	动态范围较BF16小
BF16 E8M7	16 bit	训练/推理主流格式	Exponent多，动态范围接近FP32
FP8 E4M3	8 bit	训练/推理低精度	Mantissa多，精度较好
FP8 E5M2	8 bit	训练梯度相关	Exponent多，动态范围较大
INT8	8 bit	W8A8、W8A16、KV cache	硬件成熟，工程部署稳定
INT4	4 bit	W4W16、W4A8、KV cache	压缩强，误差敏感
FP4	4 bit	前沿低比特训练/推理	动态范围比INT4灵活

↓ 量化带来的理论收益

一份数据从 BF16 → INT8，内存占用压缩 50%，算力翻倍。

实际量化收益达不到理论上限。存储上新增了量化参数；计算上新增了量化、反量化过程；并非全部数据都进行量化。

Qwen3-235B HIF4

Sglang+evalscope

量化方案	MMLU-Pro	CMMLU	MATH500	GPQA-Diamond	Average
BF16报告基准	75.2	-	91.2(456/500)	62.9(124.54/198)	
BP16自测	74.28	86.6	89.4	57.45	76.93
HiF4 attnA4W4 moeA4W4	73.27	85.82	87.2	57.07	75.84
HiF4 attnA4W4(Hadamard算法) moeA4W4	73.38	85.8	88.73	56.56	76.12
HiF4 attnA4W4(FlatQuant算法) moeA4W4	73.48	85.62	89.06	57.41	76.39

HiF4在Qwen3-235B上的精度表现

- Attn-linear和moe均采用HiF4直转，仅MATH500掉点超过1个点，MMLU-Pro接近1个点，CMMLU和GPQA-Diamond均在1个点内；
- Attn-linear采用Hadamard算法&moe采用HiF4直转，所有任务均在1个点内，平均任务掉点0.81；
- Attn-linear采用Hadamard算法&moe采用HiF4直转，所有任务均在1个点内，平均掉点掉点0.54。

量化位置与量化粒度

量化位置主要包括权重量化、激活量化、KV Cache 量化，可**组合使用**，形成不同配置

权重量化 (W)

静态

- 训练好的模型权重
- 离线完成，不影响推理流程
- 最安全的量化方式
- 典型：W8A16、W4A16

激活量化 (A)

动态

- 推理过程中的中间激活张量
- 在线完成，随输入变化
- 量化难度较大，对 outlier 敏感
- 典型：W8A8

KV Cache 量化

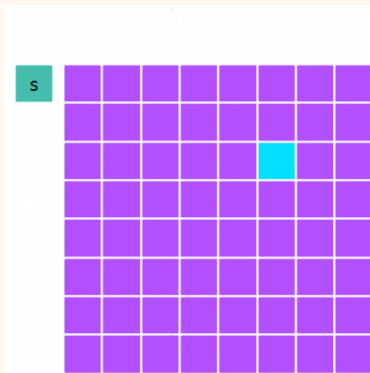
动态

- Attention中的 Key/Value
- 在线完成，随输入变化
- 随着上下文长度的增加而增加
- 典型：KV INT8

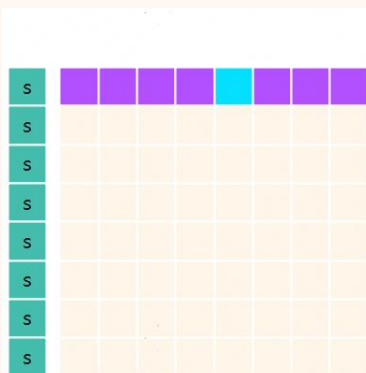
量化粒度

一组数值在量化时共享量化参数的范围大小。
粒度越细，量化误差通常越小，但需要存更多的量化参数，计算和实现也更复杂。

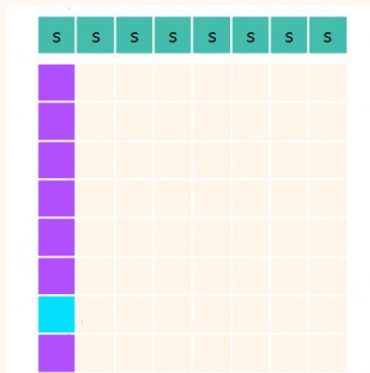
per-tensor



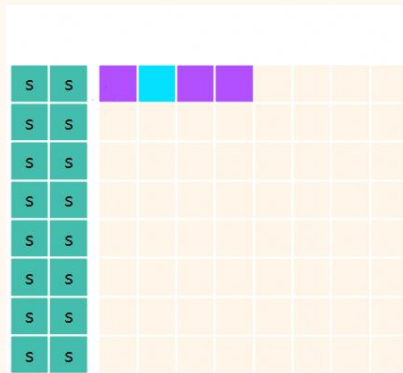
per-token



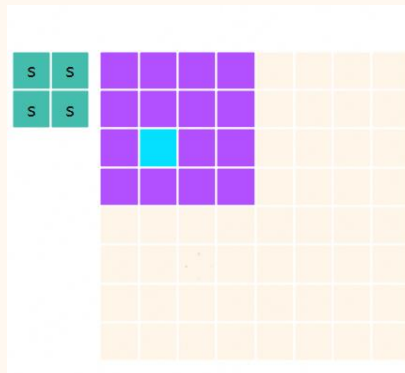
per-channel



per-group



per-block



三种量化对象独立可选，根据场景需求灵活组合。

量化映射——以对称INT8量化为例

一组FP16/BF16/FP32数据 → 一组INT8 + scale

量化 Quantize

$$scale = \frac{\max(|x|)}{127}$$
$$q = clip(round(\frac{x}{scale}), -127, 127)$$

反量化 DeQuantize

$$\hat{x} = q \times scale$$

$\hat{x}$ 是真值 x 的近似。

数值例子

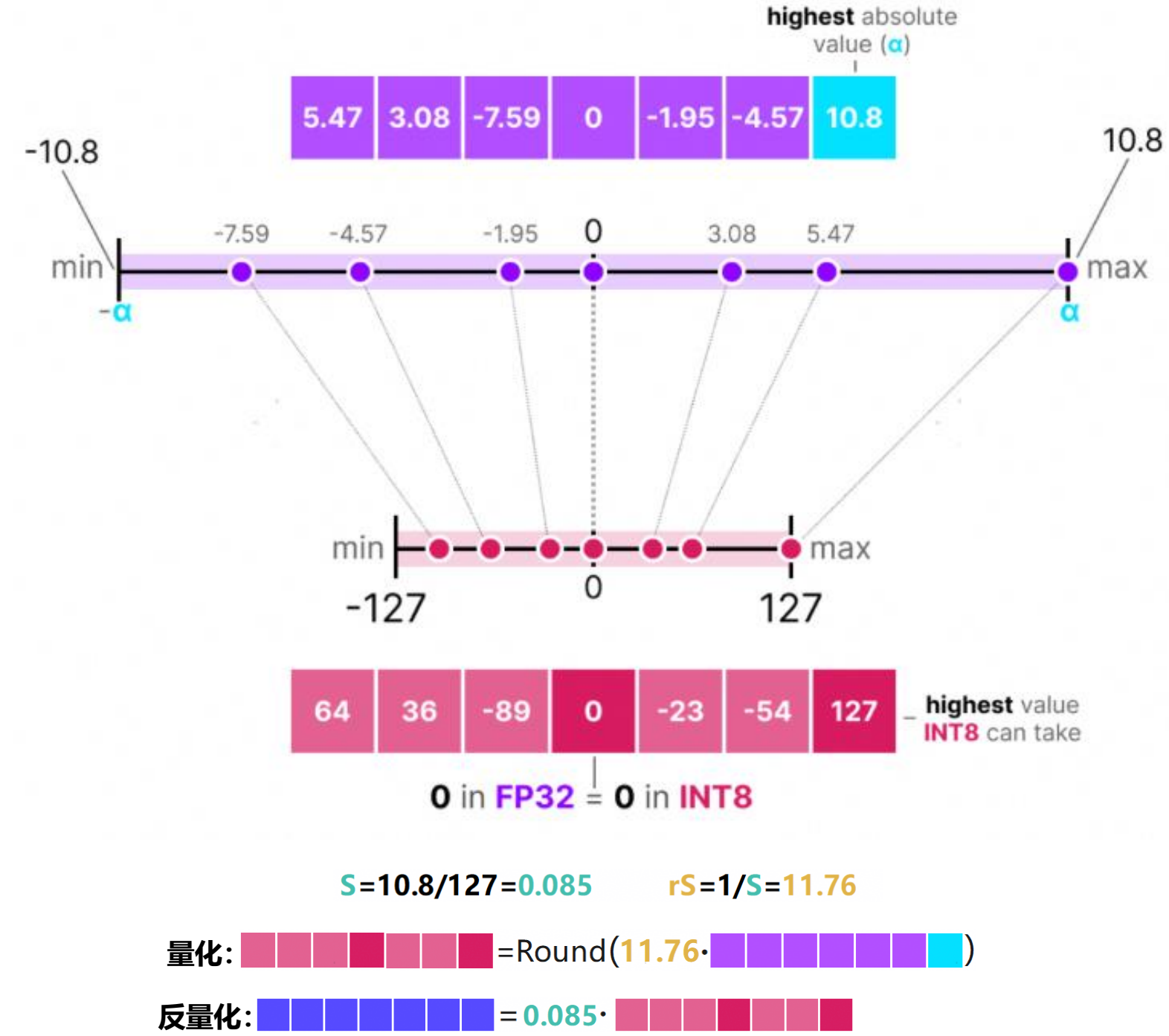
$x=[5.47, 3.08, -7.59, 0, -1.95, -4.57, 10.8]$

$\max(|x|)=10.8$

$scale = 10.8 / 127 \approx 0.085$

$q=clip(round(5.47/0.039), -127,127) = 64$

$x_hat=64\times0.085=5.44$



本例为 signed symmetric INT8，范围取 [-127, 127]；不同框架也可能使用 [-128, 127] 或非对称 zero point。

量化映射——以非对称INT8量化为例

一组FP16/BF16/FP32数据 → 一组INT8 + scale + zero_point

量化 Quantize

$$scale = \frac{\max(x) - \min(x)}{255}$$

$$zp = clip(round\left(-128 - \frac{\min(x)}{scale}\right), -128, 127)$$

$\hat{x}$ 是真值 x 的近似。

$$q = clip(round\left(\frac{x}{scale} + zp\right), -128, 127)$$

反量化 DeQuantize

$$\hat{x} = (q - zp) \times scale$$

数值例子

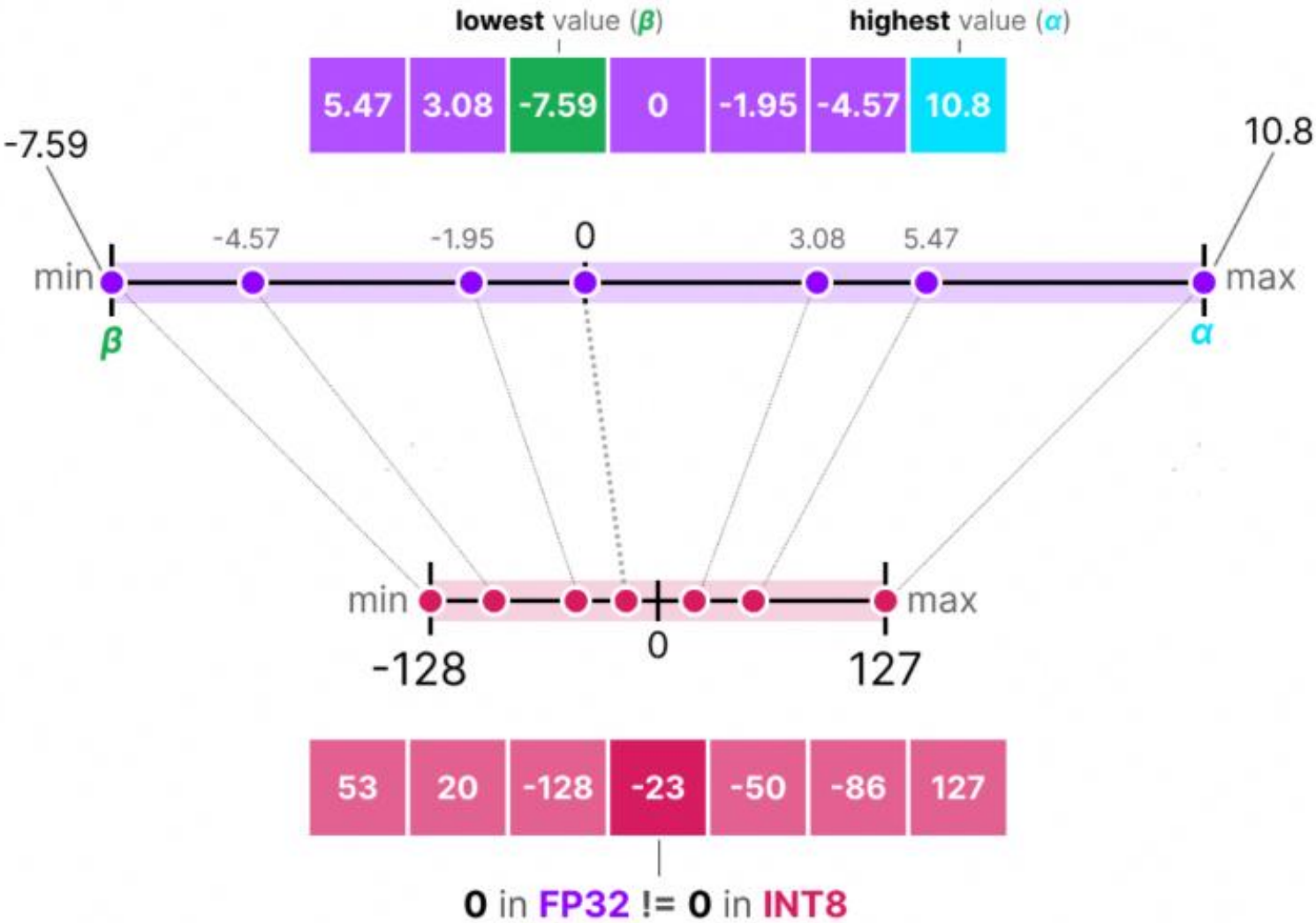
$x=[5.47, 3.08, -7.59, 0, -1.95, -4.57, 10.8]$

$\max(x)-\min(x)=10.8-(-7.59)=18.39$

$scale=18.39/255\approx0.072$, $zp=clip(round(-128+105.24), -128,127)=-23$

$q=clip(round(5.47/0.072-23), -128,127) = 53$

$x_hat=(53+23)\times0.072=5.472$



$$S=(10.8--7.59)/255=0.072 \quad Z=round(-128--105.24)=-23 \quad rS=1/S=13.87$$

量化: $\text{Round}(13.87 \cdot \text{value} - 23)$

反量化: $0.072 \cdot (\text{value} + 23)$

非对称量化适合处理数据分布明显偏向0的一侧，需要同时保存scale和zero_point参数。

量化误差的三个主要来源

Round 舍入

真实值除以 scale 后通常不是整数，只能落到最近格子。

本质：分辨率问题

影响：格子越粗，误差越大

Clip 截断

真实值超出可表示范围时，被压到边界。

本质：范围问题

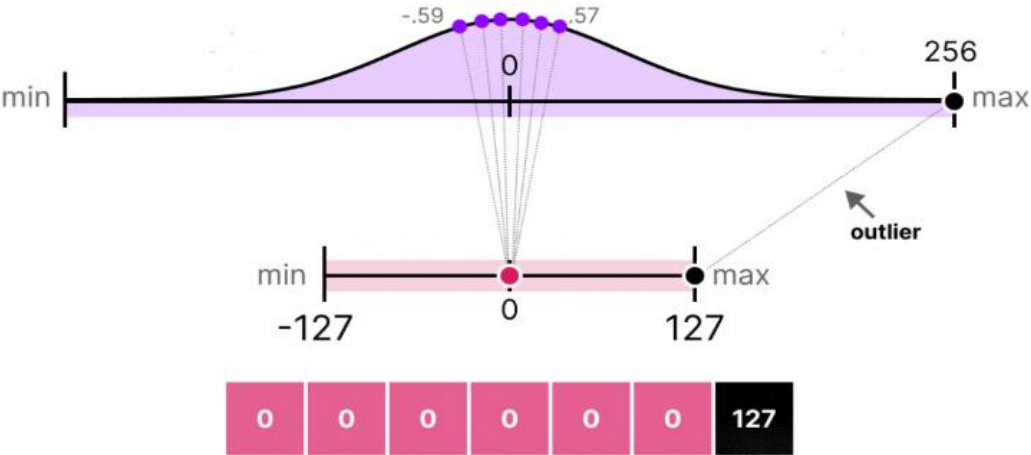
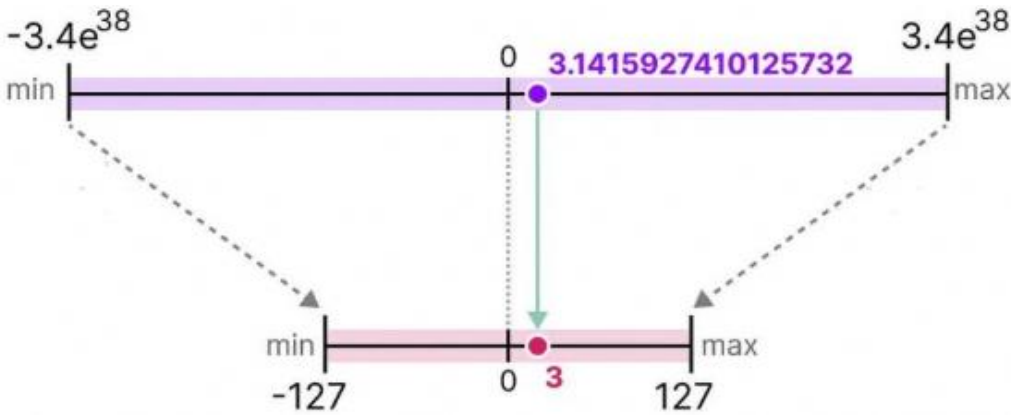
影响：范围估计太小会放大 clip

Outlier 离群值

少量极端值拉大范围，让普通值被迫使用更粗刻度。

本质：分布问题

影响：一个 outlier 可毁掉整组精度



量化算法做的很多事，本质上是在减少这三类误差对输出质量的影响。

03

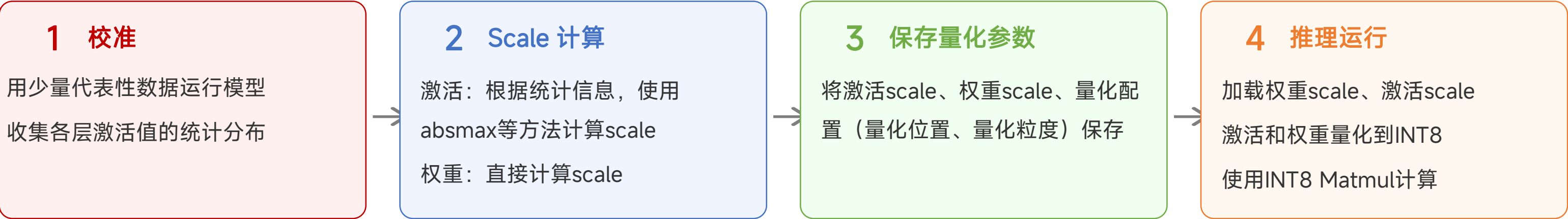
W8A8 量化详解

当前工业界最成熟的量化方案

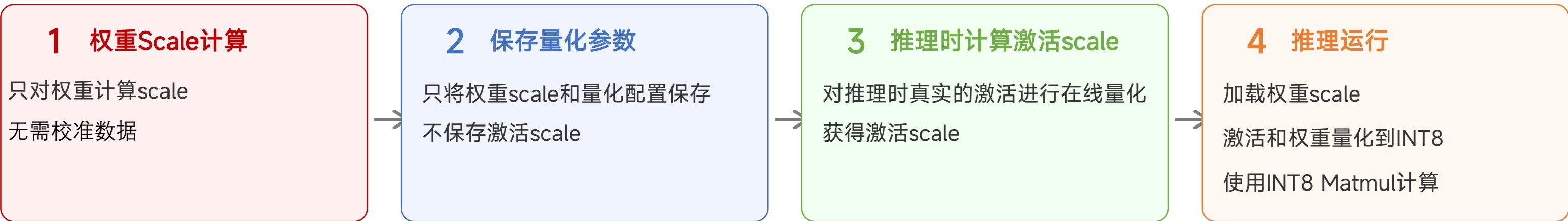
W8A8 量化原理

W8A8 = 权重 INT8 × 激活 INT8，工业界最广泛部署的量化方案

静态量化



动态量化



静态量化中，校准数据质量直接影响量化精度——应覆盖模型实际处理的主要场景和输入分布。
动态量化比静态量化精度更高，但每次推理时有额外的计算代价。



量化算法——SmoothQuant

解决激活值中的`outlier`问题，让 W8A8 精度更接近 FP16。x=[0.47, 0.08, -0.59, 0, -0.95, -0.57, **100.8**]

⚠ 问题：激活 Outlier

少数激活值极大（outlier），导致大部分数值被压缩到很小的 INT8 范围，精度严重损失。

✓ 思路：平滑迁移

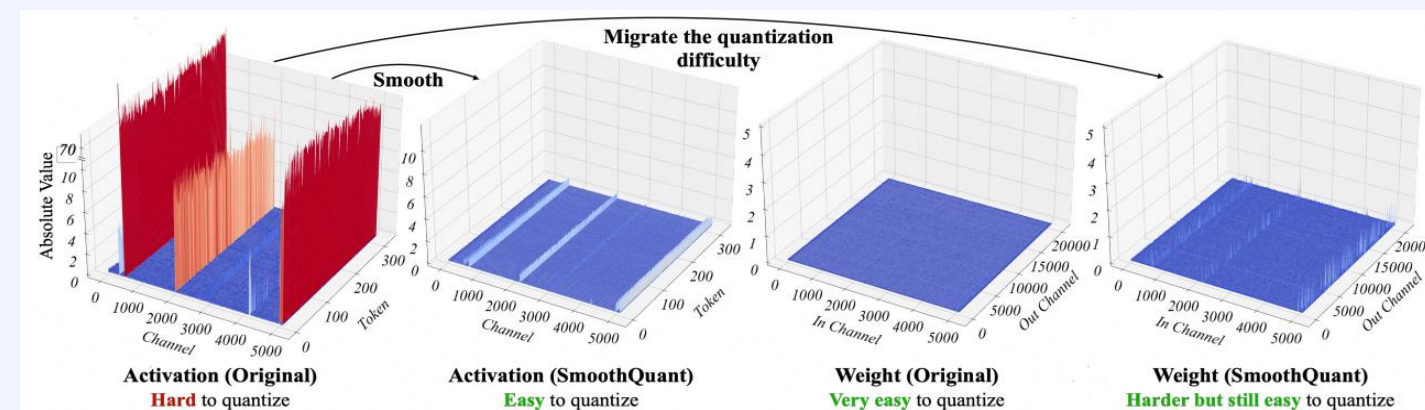
将激活的 outlier "平滑"迁移到权重上，通过数学等价变换实现，不改变输出结果。

SmoothQuant 核心公式

$$Y = X \cdot W = (X / s) \cdot (s \times W) = X' \cdot W'$$

其中 s 是对角矩阵，变换后的X' 更易量化，变换后的W' 仍保持好精度

$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}$$



效果：SmoothQuant 使 W8A8 在 LLM 上的精度接近 FP16，成为当前最可靠的 PTQ（Post-Training Quantization）方案之一

SmoothQuant论文：<https://arxiv.org/pdf/2211.10438>

<https://gitcode.com/cann>

PTQ与QAT

PTQ (Post-Training Quantization) : 训练后量化, 模型参数已经固定, 不再重训。通过增加量化参数的方式, 把模型转换成低bit形式。

QAT (Quantization Aware Training) : 量化感知训练, 在训练或者微调过程中模拟量化误差, 让模型主动适应低bit格式。

方法	发生阶段	核心做法	适用场景
PTQ	训练完成后	校准 + 离线转换 + 验证	快速部署, 成本低
QAT	训练/微调中	前向模拟量化, 反向更新参数	更低 bit (4bit) 或高精度要求

PTQ 适合快速部署; QAT 成本更高, 但在更低 bit 或高精度要求下更有优势。

精度与性能的权衡

量化带来**收益**的同时也产生**代价**，需根据场景选择策略

↑ 收益

- 显存占用降低
- 内存带宽需求降低
- 吞吐量提升
- 推理时延降低

↓ 代价

- 模型精度损失（通常 <1%）
- 需要校准数据和量化流程
- 极低比特可能显著降质
- 部分硬件和算子不支持低精度

场景化策略选择

场景	推荐方案	理由
精度敏感（对话、代码）	W8A8	精度损失 <1%，几乎无感知
速度敏感（边缘、高并发）	W4A16	权重压缩 4 倍，吞吐最大化

量化并不是将数据bit压的越低越好，而是要综合考虑实际的量化开销、精度损失，是收益与性能平衡的艺术。

04

主流量化路线对比

GPTQ、AWQ、FP8 与更多低比特方案

量化算法演进路线

从最简单的四舍五入到复杂的误差补偿，量化算法持续**演进**

算法	核心思想	技术特点	适用场景
RTN	Round-To-Nearest	直接四舍五入到最近整数	基线方法，精度最低
GPTQ	逐层量化，误差累积补偿	OBQ 优化，逐层更新权重	权重量化，W4 场景
AWQ	保护重要权重通道	基于激活幅度识别 salient 权重	权重量化，精度更优
SmoothQuant	迁移激活 outlier 到权重	数学等价变换， X/s 和 $s \times W$	W8A8，工业主流
QuaRot	旋转激活空间消除 outlier	Hadamard 旋转，更难量化	4bit量化前沿研究方向



趋势：从简单舍入 → 误差补偿 → 分布变换，精度持续提升

SmoothQuant 因其稳定性和成熟度，成为当前工业界 W8A8 的首选方案。

低比特格式前沿

MXFP8、MXFP4 等新兴格式在**硬件原生支持**下成为新趋势

FP8 (E4M3/E5M2)

- 8 位浮点，两种变体
- E4M3: 1+4+3, 精度优先
- E5M2: 1+5+2, 范围优先
- 适合训练场景

MXFP8

- Microscaling FP8
- Block-wise scaling
- 比 per-token/per-channel FP8 更精细
- 精度优于标准 FP8

MXFP4

- Microscaling FP4
- Block-wise scaling
- 需要联合量化算法协同降低精度损失
- 基本全面取代FP4

硬件支持现状

- 昇腾 A2/A3: 原生支持 INT8、FP8; 昇腾 A5: 原生支持 HiF8、MXFP8、MXFP4; 昇腾 A6: 原生支持 HiF4

低比特格式的选择需综合考虑硬件支持、精度要求和部署场景。

05

CANN 仓库实践

在 cann-recipes-infer 中落地量化

<https://gitcode.com/cann>

CANN

Qwen3-8B W8A8 量化实践

结合 [cann-recipes-infer](#) 仓库中的配置，理解量化实践

使用昇腾官方大模型量化工具 [AMCT \(Ascend Model Compression Toolkit\)](#)，可通过命令行直接导出 Qwen3-8B W8A8 INT8 部署权重。

导出的内容如下：

项	要求
格式	compressed-tensors HuggingFace 风格目录，包含 <code>layer_*.safetensors</code> 、 <code>rest_*.safetensors</code> 、更新后的 <code>model.safetensors.index.json</code> 与 <code>config.json</code> （内嵌 <code>quantization_config</code> ）
量化模式	W8 INT8 per-channel weight + A8 INT8 dynamic activation
量化范围	attention 4 线性层（q/k/v/o_proj）+ MLP（gate_proj / up_proj / down_proj）全量化
ignore 列表	使用 HF 风格非融合层名（q_proj / k_proj / v_proj / gate_proj / up_proj），框架侧 <code>should_ignore_layer</code> 已通过 <code>fused_mapping</code> 将其映射到融合层（merged_qkv_proj / gate_up_proj）

act / weight 量化模式： 激活/权重的数据类型、量化粒度、静态动态

include / ignore： 指定/排除量化位置

save： 量化产物（safetensors）保存路径

 配置位于 `models/qwen/config/` 目录下，与推理配置分离管理

<https://gitcode.com/cann/cann-recipes-infer/blob/master/models/qwen/README.md>



infer仓内量化运行链路

量化配置与推理运行时衔接，形成完整的**量化推理闭环**

Linear W8A8

- 标准线性层量化
- $W_{int8} \times A_{int8}$ GEMM
- compressed-tensors 格式
- 最基础的量化路径

MoE GMM

- MoE 专家层量化
- Grouped MatMul
- Expert 权重独立量化
- 路由逻辑保持不变

KV Cache INT8

- KV Cache 量化存储
- 计算时反量化
- 长序列显存优化
- 对生成质量影响小

统一入口: `compressed_tensors.py`

仓内标准量化入口识别 `compressed-tensors` 格式的量化产物

量化配置通过 `config.json` 中的 `quantization_config` 字段与运行时衔接

运行时根据配置自动选择 Linear / MoEGMM / KV Cache 三类量化路径

完整的量化推理闭环 = 量化配置 + 量化产物 + 运行时适配。→ gitcode.com/cann/cann-recipes-infer

Thank you.

开启你的量化优化之旅

gitcode.com/cann/cann-recipes-infer

核心要点回顾

- 量化是降低推理成本的关键手段
- W8A8 是当前最成熟的工业方案
- SmoothQuant 解决激活 outlier 问题
- CANN 生态提供完整量化工具链
- 读懂配置 → 定位链路 → 实践验证



上CANN社区获取干货



关注CANN公众号获取资讯